PROJECT HEALTH REPORTING AGENT

An AI Agent for Automated RAG Status Reporting Across Project Portfolios

Technical Assignment — AI Engineer Intern, Zycus

Prepared by: Janakiraman K
janakiramankeerthivelan1021@gmail.com
10 July 2026

Table of Contents

Table of Contents 2
1. Executive Summary 4
What Was Delivered 4
2. Problem Statement and Approach 4
2.1 The Problem 4
2.2 The Approach 4
3. RAG Methodology (Phase 1 Deliverable) 5
3.1 Weighted Scoring Formula 5
3.2 RAG Thresholds 5
3.3 Component Formulas in Detail 6
Schedule Health — 30% 6
Budget Burn — 20% 6
Milestone Health — 20% 6
Risks — 10% 6
Blockers — 10% 6
Stakeholder Sentiment — 5% 6
Data Quality — 5% 7
3.4 Assumptions 7
4. System Architecture 7
4.1 Design Philosophy 7
4.2 Pipeline Overview 7
4.3 End-to-End Execution Flow 8
4.4 Module Reference 9
run.py 9
src/cli.py 9
src/config.py 9
src/loader.py 9
src/preprocess.py 9
src/validator.py 10
src/metrics.py 10
src/scoring.py 10
src/rag_engine.py 10
src/insights.py 10
src/charts.py 10
src/report_generator.py 10

src/ppt_generator.py 10
src/scheduler.py 11
src/llm_reasoning.py 11
5. Handling Incomplete or Messy Data 11
6. Sample Results — Weekly and Monthly Outputs 11
6.1 Portfolio Summary 11
6.2 Zycus – UniSan S2P Implementation 12
6.3 Zycus – Titan S2P Implementation 12
6.4 Recommendations Generated by the Agent 13
6.5 Deliverable Files Produced 13
7. Key Design Decisions 13
8. Running the Agent and Scheduling (Bonus) 14
8.1 Setup 14
8.2 Run Modes 14
8.3 Automation Reference (Bonus Requirement) 14
8.4 Testing 14
9. Future Improvements 14
10. Closing Note 15

1. Executive Summary

Leadership at Zycus needs weekly visibility into project health without manually chasing project managers for status. The Project Health Reporting Agent is a Python-based system that reads raw project-plan workbooks, independently computes a Red/Amber/Green (RAG) status for each project from task-level evidence, explains the reasoning in plain English, and produces both weekly reports and a monthly executive presentation — with no manual data wrangling required.

The agent was built and tested end-to-end against the two sample workbooks supplied with the assignment (S2P Project.xlsx and Project Plan B.xlsx). On this data, the agent rates the overall portfolio RED at a score of 59.4, and — notably — computes an AMBER status for a project whose workbook self-reports GREEN, because task-level evidence (overdue work, Red/Amber task signals, incomplete critical tasks and blocker comments) tells a different story. This discrepancy-detection behaviour is the core value proposition of the system: it stops leadership from unknowingly trusting an optimistic self-reported status.

What Was Delivered

- A one-page RAG methodology defining exactly how schedule, budget, milestones, blockers and sentiment map to a RAG status (Section 3).
- A working AI agent — reads a project plan, determines RAG status, explains its reasoning in plain English, and handles messy/incomplete data gracefully (Sections 4–5).
- Sample weekly outputs for each supplied project plan, in Markdown, HTML, JSON and TXT (Section 6).
- A monthly synthesis: a 7-slide executive PowerPoint that identifies trends across projects, highlights emerging risks, and gives recommendations (Section 6).
- A README and this document explaining every design decision (Section 7).
- Bonus: a weekly/monthly scheduling reference so the agent can be automated (Section 8).

2. Problem Statement and Approach

2.1 The Problem

Project plan exports are inconsistent by nature: sheet names differ between workbooks, columns are named differently across teams, dates and completion percentages are sometimes blank, comments live on separate tabs, and — most importantly — the project manager's self-reported RAG colour does not always agree with what the underlying task data actually shows. Leadership currently has to manually read each workbook every week to get a true picture.

2.2 The Approach

Rather than simply summarising whatever the workbook already says, the agent treats the workbook-entered status as one input signal, not the answer. It independently derives a RAG status from task-level evidence — schedule progress vs. elapsed time, overdue and critical-path tasks, blocker language in comments — and then explicitly compares its computed status against

the workbook's self-reported status. Any disagreement is surfaced to leadership as a discrepancy, which is exactly the kind of blind spot a manual weekly check is likely to miss.

This is implemented as a small, testable pipeline (not a single monolithic script), so each concern — reading data, cleaning it, validating it, scoring it, explaining it, and presenting it — is isolated in its own module. That keeps the reasoning auditable: every number in the final report can be traced back to a specific task-level signal.

3. RAG Methodology (Phase 1 Deliverable)

This section is the one-page methodology requested in Phase 1 of the assignment, reproduced here alongside the rest of the write-up (also delivered standalone as methodology.md in the submission).

3.1 Weighted Scoring Formula

Final health is a single weighted score from 0 to 100, computed from seven components:

```
Final Score = Schedule(0.30) + Budget(0.20) + Milestones(0.20) + Risks(0.10) + Blockers(0.10) + Sentiment(0.05) + Data Quality(0.05)
```

Component	Weight	What it measures
Schedule Health	30%	Actual completion vs. expected completion given elapsed time; overdue tasks; Red task signals; zero-float work.
Budget Burn	20%	Budget/cost/spend fields when present; neutral score with low confidence when absent.
Milestone Health	20%	Tasks due soon, incomplete critical tasks, and overall schedule gap.
Risks	10%	Red/Amber task-level signals, incomplete critical tasks, on-hold tasks.
Blockers	10%	Blocker-style stakeholder comments, on-hold tasks, zero-float incomplete work.
Stakeholder Sentiment	5%	Deterministic keyword analysis of comments (positive vs. risk language).
Data Quality	5%	Penalises missing columns, invalid dates, inferred values, unknown statuses.

3.2 RAG Thresholds

Status	Score Range	Meaning
GREEN	85 – 100	On track; no material schedule, risk or blocker concerns.
AMBER	60 – 84	Caution; some evidence of schedule slippage, risk or blockers.
RED	0 – 59	At risk; strong evidence of overdue work, critical-path exposure, or unresolved blockers.

3.3 Component Formulas in Detail

Schedule Health — 30%

Expected completion is calculated as elapsed calendar days divided by total planned project days; the schedule gap is actual completion minus expected completion. The score starts at 100 and is penalised for a negative schedule gap, overdue incomplete tasks, task rows with a Red health signal, and incomplete zero-float (critical-path) tasks.

Budget Burn — 20%

Uses budget, cost, spend or burn-rate columns when the workbook provides them. Neither supplied sample workbook contains financial fields, so the agent reports a neutral score of 70 with low confidence rather than guessing a financial conclusion it cannot support — an explicit design choice covered in Section 7.

Milestone Health — 20%

Combines tasks due within a 14-day lookahead window, incomplete critical tasks, and the overall schedule gap. Where explicit milestone flags are sparse, the agent falls back to task hierarchy, dates, completion percentage and critical-path fields.

Risks — 10%

Penalises Red/Amber task-level signals, incomplete critical tasks, and on-hold tasks — surfacing risk even when a workbook's own summary looks healthy.

Blockers — 10%

Derived from on-hold tasks, incomplete zero-float work, and comment text scanned for operational blocker language (for example: pending, impacted, waiting, delay, hold, need, remain).

Stakeholder Sentiment — 5%

A deterministic, keyword-based read of stakeholder comments — completion-oriented language raises the score, risk or unresolved language lowers it. Sentiment is neutral with low confidence when no comments exist.

Data Quality — 5%

Starts at 100 and is reduced for missing required columns, missing optional signals, invalid dates, inferred (rather than reported) completion values, and unrecognised status values.

3.4 Assumptions

- Completion values may appear as 0.44, 44, or blank; blank completion is inferred from the task's status field.
- The workbook value "Yellow" is normalised to AMBER throughout the pipeline.
- Missing budget data reduces confidence — it never forces a RED or GREEN financial conclusion.
- Critical tasks, zero-float tasks, overdue tasks, comments and schedule gap are treated as stronger evidence than the workbook's own summary status.
- Where present, the workbook's own "Today's Date" field is used as the as-of date, so re-running the agent on the same file reproduces the same report.

4. System Architecture

4.1 Design Philosophy

The system is built as a small, production-style pipeline. Each stage has exactly one responsibility and passes a clean, structured object to the next stage — so any single concern (a new workbook format, a new chart, a new report format) can be extended without touching unrelated code.

4.2 Pipeline Overview

Excel workbooks in data/ → loader (sheet discovery) → preprocess (canonical schema) → validator (data-quality checks) → metrics (task-level signals) → scoring (weighted RAG) → rag_engine (project & portfolio objects) → insights (executive narrative) + charts (visuals) → report_generator (Markdown / HTML / JSON / TXT) + ppt_generator (executive deck).

Stage	**Module**	Responsibility
1	loader.py	Scans data/ for .xlsx workbooks; profiles every sheet to auto-detect task sheets, summary sheets and comment sheets (no fixed sheet names assumed).
2	preprocess.py	Maps inconsistent workbook columns onto a canonical schema (task_name, status, percent_complete, dates, RAG, owner, comments, etc.) using alias lists.

Stage	Module	Responsibility
3	validator.py	Checks required/optional fields; records data-quality findings instead of crashing on missing data.
4	metrics.py	Computes task-level evidence: overdue counts, completion gap, due-soon tasks, Red/Amber signals, incomplete critical tasks, zero-float tasks, blocker comments, missing owners.
5	scoring.py	Converts metrics into a weighted 0-100 health score and maps it to RAG using configured thresholds.
6	rag_engine.py	Orchestrates stages 1-5 per workbook and aggregates results into a portfolio-level object (ProjectRun / PortfolioRun).
7	insights.py	Converts numeric results into plain-English executive narrative: summary, risks, blockers, trends, recommendations.
8	charts.py	Generates PNG charts (health mix, comparison, trend, risk heat-map) from the same evaluation object.
9	report_generator.py	Writes weekly reports in Markdown, HTML, JSON and TXT — portfolio-level and per-project.
10	ppt_generator.py	Builds the 7-slide monthly executive PowerPoint for VP/client presentation.

4.3 End-to-End Execution Flow

What happens internally when python run.py is executed:

1. run.py calls src.cli.main().
2. cli.py loads runtime settings from config/default_config.json (scoring weights, RAG thresholds, lookahead window).
3. Required output folders are created if they do not already exist.
4. scheduler.py writes a human-readable schedule reference file for later automation.
5. rag_engine.py begins workbook evaluation.
6. loader.py scans data/ for .xlsx / .xlsm workbooks and profiles every sheet using column-name evidence, row counts, and key/value patterns to classify it as a task, summary, or comment sheet.
7. preprocess.py maps workbook-specific column names onto a canonical schema (task_name, status, percent_complete, start_date, end_date, schedule_health, rag, critical, total_float, assigned_to, comments).

8. validator.py checks for required and optional fields, recording data-quality findings rather than halting execution.
9. metrics.py computes task-level signals: overdue work, completion percentage, due-soon tasks, Red/Amber signals, incomplete critical tasks, zero-float tasks, blocker comments, missing owners.
10. scoring.py converts those signals into the weighted 0–100 health score and RAG status described in Section 3.
11. rag_engine.py aggregates every project's result into a single portfolio-level object.
12. insights.py converts the numeric result into an executive narrative: summary, risks, blockers, trends, recommendations.
13. charts.py renders the supporting chart images.
14. report_generator.py writes the Markdown, HTML, JSON and TXT reports (portfolio-level and per-project).
15. ppt_generator.py builds the 7-slide monthly executive PowerPoint.
16. Execution logs are written to logs/.

4.4 Module Reference

run.py

A single-line entry point (from src.cli import main) so reviewers can run the whole system with one command: python run.py.

src/cli.py

Owns the command-line interface: --weekly, --monthly, --ppt, --json, --html, --data-dir, --as-of, --llm-provider. Its run_agent() function coordinates the full workflow — load settings, configure logging, evaluate workbooks, generate charts, generate reports, generate the PowerPoint, and print a completion summary.

src/config.py

Loads the scoring weights and RAG thresholds from config/default_config.json, so the methodology in Section 3 is configuration, not a hard-coded constant — weights and thresholds can be tuned without touching the scoring logic.

src/loader.py

Discovers and reads workbooks without assuming fixed sheet names. A sheet with columns like Task Name, Status, % Complete, Start Date and End Date scores high as a task sheet; a compact sheet with labels like Project Manager, Project Stage and Schedule Health scores high as a summary sheet. This is what lets the same pipeline handle workbooks exported in slightly different formats.

src/preprocess.py

Converts messy workbook data into the clean canonical dataset used everywhere downstream — normalising column-name aliases (e.g. Task / Activity → task_name), parsing dates and float

values, inferring completion when blank, extracting comments from both comment sheets and inline task notes, and pulling project metadata from the summary sheet.

src/validator.py

Checks for required fields (task_name, status, percent_complete, start_date, end_date) and important optional fields (schedule_health, rag, critical, total_float, variance, assigned_to, comments, phase). Findings are surfaced in the report so a reviewer can see exactly where confidence is strong or weak, rather than the system silently guessing.

src/metrics.py

The evidence layer: computes total/active tasks, average and expected completion, schedule gap, overdue incomplete tasks, tasks due in the lookahead window, Red/Amber task counts, incomplete critical tasks, zero/negative-float tasks, on-hold tasks, blocker-like comments, and missing-owner counts.

src/scoring.py

Applies the weighted formula from Section 3 to the metrics above. Crucially, the workbook's own Schedule Health / RAG fields are used only as comparison signals — the computed RAG is derived independently from task-level evidence, never copied from the workbook.

src/rag_engine.py

The orchestrator: connects workbook loading, preprocessing, validation, metrics and scoring, and returns structured ProjectRun and PortfolioRun objects that every downstream module (reports, charts, PPT) reuses — so every output is guaranteed to show consistent numbers.

src/insights.py

Turns the numeric result into business-readable language: executive summaries, major risks, recommendations, top blockers, resource constraints and emerging trends — the layer that makes the output useful to a VP, not just correct to an engineer.

src/charts.py

Generates the supporting PNG charts with Matplotlib: portfolio health mix, project health comparison, planned-vs-actual completion trend, risk heat-map, and completion-vs-expected progress — saved to outputs/charts/.

src/report_generator.py

Writes the weekly portfolio report and per-project reports in Markdown, HTML, JSON and TXT. The JSON output preserves the full structured evaluation payload for downstream automation or dashboards.

src/ppt_generator.py

Builds the monthly executive PowerPoint (presentations/monthly_portfolio_health.pptx) with seven slides: title & portfolio status, portfolio health overview, project comparison, emerging risks, recommendations, evidence charts, and next steps.

src/scheduler.py

Writes ready-to-use weekly and monthly automation commands to config/schedule_reference.txt (Bonus deliverable — see Section 8).

src/llm_reasoning.py

An optional LLM adapter (OpenAI, Groq or Ollama) for enhanced natural-language reasoning. The system never depends on it: in mock mode — the default — every output is fully deterministic and reproducible with no API key required, and if a configured provider's call fails, the deterministic fallback is used automatically.

5. Handling Incomplete or Messy Data

This was an explicit requirement of Phase 2 ("Handles incomplete or messy data gracefully"), so it is treated as a first-class concern rather than an edge case:

- Sheet discovery does not depend on fixed sheet names or column order — each sheet is scored on evidence and classified as task / summary / comment.
- Column mapping uses alias lists, so equivalent columns named differently across workbooks (Task / Activity, % Complete / Percent Complete / Completion) still resolve to the same canonical field.
- Missing required fields do not stop the run; they are recorded as validation findings and reflected as a lower data-quality score and confidence value.
- Blank completion percentages are inferred from the task's status field rather than left null.
- Missing budget/financial columns produce a neutral, low-confidence Budget Burn score instead of a fabricated conclusion.
- Every report explicitly states its confidence percentage, so leadership knows how much to trust a given number.

6. Sample Results — Weekly and Monthly Outputs

The figures below are the actual outputs produced by running python run.py --llm-provider mock against the two supplied workbooks (S2P Project.xlsx and Project Plan B.xlsx), as of 2026-07-08.

6.1 Portfolio Summary

Metric	Portfolio	UniSan S2P	Titan S2P
Computed RAG	RED	RED	AMBER
Workbook RAG	—	RED	GREEN
Computed Score	59.4	56.96	61.9
Confidence	—	62%	70%
Completion	51%	44% (exp. 59%)	57% (exp. 57%)

Metric	Portfolio	UniSan S2P	Titan S2P
Overdue incomplete tasks	44	25	19
Incomplete critical tasks	46	31	15
Blocker-style comments	6	—	6

Overall, the portfolio is rated RED at a score of 59.4, spanning 876 tasks across 2 projects at 51% weighted completion. Leadership attention should focus on 44 overdue incomplete tasks, 46 incomplete critical tasks, and 6 blocker-style stakeholder comments.

6.2 Zycus – UniSan S2P Implementation

Computed RAG: RED (score 56.96, confidence 62%) — agrees with the workbook's own RED status. 44% complete against an expected 59%, with 25 overdue incomplete tasks and 31 incomplete critical tasks.

Component	Score	Weight	Key Evidence
Schedule Health	46.61	30%	44% complete vs 59% expected; 25 overdue tasks; 42 Red signals.
Budget Burn	70.0	20%	No budget fields in source; neutral score, low confidence.
Milestone Health	36.04	20%	31 incomplete critical tasks; large schedule gap.
Risks	54.01	10%	42 task-level Red signals across the workbook.
Blockers	87.47	10%	Few blocker-style comments recorded.
Stakeholder Sentiment	75.0	5%	Neutral-to-positive comment language.
Data Quality	77.5	5%	Some optional fields missing; core fields present.

6.3 Zycus – Titan S2P Implementation

Computed RAG: AMBER (score 61.9, confidence 70%) — this disagrees with the workbook's self-reported GREEN status. 57% complete against an expected 57% (on pace), but 80 task-level Red signals, 19 overdue tasks and 15 incomplete critical tasks are enough evidence for the agent to downgrade the status. Top blocker comments include pending JDE field mapping and unscheduled onsite workshop coordination.

Component	Score	Weight	Key Evidence
Schedule Health	74.31	30%	57% complete vs 57% expected; 19 overdue tasks.
Budget Burn	70.0	20%	No budget fields in source; neutral score, low confidence.
Milestone Health	62.98	20%	15 incomplete critical tasks.
Risks	17.21	10%	80 task-level Red signals — the main driver of the AMBER rating.
Blockers	37.42	10%	6 blocker-style comments: JDE mapping pending, workshop scheduling.
Stakeholder Sentiment	60.0	5%	Mixed comment language.
Data Quality	91.01	5%	Well-populated workbook.

This Titan discrepancy is the clearest illustration of why the agent exists: a workbook that self-reports GREEN is quietly carrying more task-level risk than a workbook that honestly reports RED. A manual weekly check that trusts the workbook's own colour would miss this.

6.4 Recommendations Generated by the Agent

- Create a weekly exception review focused on overdue, critical and zero-float tasks.
- Normalise workbook hygiene across projects — particularly owner, milestone, budget and comment fields.
- Separate self-reported workbook health from independently computed health in leadership reporting.
- Add budget-burn data to the standard project-plan export so health scoring is not schedule-only.

6.5 Deliverable Files Produced

- methodology.md — one-page RAG methodology.
- reports/weekly_report.{md,html,json,txt} — portfolio-level weekly report.
- reports/weekly_<project>.{md,html,json,txt} — per-project weekly reports.
- outputs/portfolio_health.json — full structured evaluation payload.
- outputs/charts/*.png — health mix, comparison, trend and risk-heatmap charts.
- presentations/monthly_portfolio_health.pptx — 7-slide executive deck.
- logs/agent_run.log, logs/sample_execution.log — execution logs.

7. Key Design Decisions

- Health is computed independently and never copied from the workbook — the workbook's own status is used purely as a comparison signal, which is what makes discrepancy detection possible.
- Missing financial data lowers confidence rather than inventing a Green or Red budget conclusion.
- Messy or incomplete data is reported through validation findings instead of stopping execution — the agent always produces an answer, with an honest confidence score attached.
- Column mapping uses alias lists rather than fixed column positions, so the same pipeline tolerates workbook-format drift.
- Every report, chart, JSON export and the PowerPoint are all generated from the same PortfolioRun object, so numbers can never disagree across output formats.
- mock LLM mode is deterministic and requires no API key, so the submission is fully reproducible by a reviewer with zero external credentials; OpenAI, Groq and Ollama are supported as optional upgrades for richer narrative text.

8. Running the Agent and Scheduling (Bonus)

8.1 Setup

```
python -m venv .venv .venv\Scripts\activate python -m pip install --upgrade pip python -m pip install -r requirements.txt
```

8.2 Run Modes

```
python run.py # full run: reports + charts + PPT python run.py --weekly # weekly reports only python run.py --monthly # monthly synthesis only python run.py --ppt # PowerPoint only python run.py --json / --html # single-format export python run.py --data-dir data --as-of 2026-07-02 --llm-provider mock
```

Supported --llm-provider values: mock (deterministic, no credentials — the default), openai (OPENAI_API_KEY), groq (GROQ_API_KEY), and ollama (local service). The core scoring and reporting logic never requires external credentials.

8.3 Automation Reference (Bonus Requirement)

scheduler.py writes ready-to-use commands to config/schedule_reference.txt, which can be dropped directly into cron, Windows Task Scheduler, or a GitHub Actions workflow:

```
# Weekly python run.py --weekly --json --html # Monthly python run.py --monthly --ppt
```

8.4 Testing

A pytest suite (tests/) covers sheet-discovery logic, RAG threshold mapping, validation behaviour for missing fields, and plain-text report formatting:

```
python -m pytest
```

9. Future Improvements

- Add real budget-burn scoring once project exports consistently include budget, spend or burn-rate fields.
- Add dependency-graph analysis when predecessor fields are consistently maintained across workbooks.
- Persist historical runs to support genuine trend analysis over multiple weeks/months, not just a single snapshot.
- Add a lightweight web dashboard for interactive portfolio exploration.
- Add stronger natural-language comment classification using an LLM provider when credentials are available, layered on top of the existing deterministic keyword baseline.

10. Closing Note

The complete source code, sample outputs, methodology document, README, logs and this write-up are included in the submitted ZIP file. The system was designed so that every claim in the final report — every RAG status, every score, every risk statement — can be traced back to a specific, inspectable piece of task-level evidence in the source workbook.